pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Statement Sentiment

WEEK 4 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The problem: can wording become a number?

The 46 FOMC statements, 2021-01-27 to 2026-09-16

46
published statements

10,204
tokens after cleaning

2,940
daily yield rows

Statements: Board of Governors of the Federal Reserve System,
Federal Open Market Committee statements, federalreserve.gov.
Retrieved on 2026-10-03. The text in the file is exactly as published.
Yields: U.S. Department of the Treasury, Daily Treasury Par Yield Curve Rates,
retrieved on 2026-10-03, in percent, as published.

```
statements = pd.read_csv(data_folder + 'fomc_statements.csv')
statements['kept'] = statements['text'].apply(clean).apply(remove_stop_words)
print(statements['n_kept'].sum())   # 10204
```

Every number is printed beside the words that produced it. Every quotation carries its date.

The desk head asks

1. What do two short word lists give?
2. Which words produce that score?
3. What does a published lexicon give?
4. And a list built for this text?
5. Do the three scores agree?
6. Any relation to changes in yields?

A score is a property of the wording, measured by a stated word list. It does not say what the authors meant or what happens next, and it is not a reason to do anything.

Both files are real and public. There is no portfolio data. Nothing here is a view on policy, on rates, or on any security.

A score from scratch

Two lists typed by hand, 14 general words each

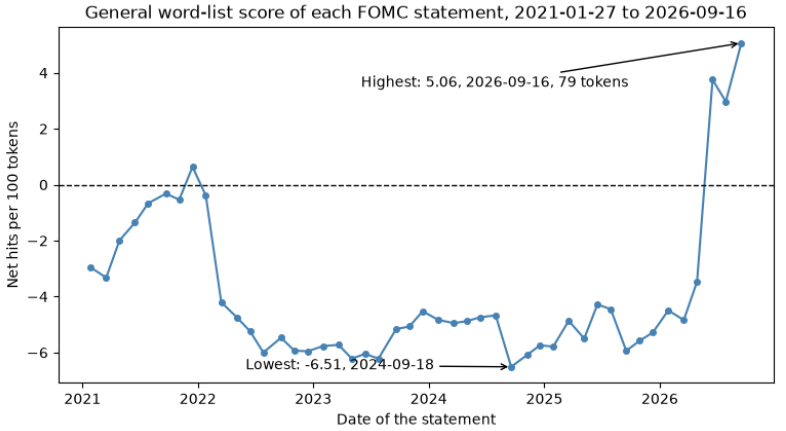
Positive `good strong better gains support progress improved improvement confidence growth solid stable recovery benefit`

Negative `bad weak worse risks inflation unemployment pressures restrictive uncertain hardship decline loss debt lower`

Step	The first statement, dated 2021-01-27
Positive hits	5: support 2, progress 2, recovery 1
Negative hits	14: inflation 10, risks 2, hardship 1, pressures 1
Net	5 - 14 = -9
Per 100 tokens	-9 / 304 kept tokens * 100 = -2.96

```
positive_hits - negative_hits   # TypeError: list minus list
net = len(positive_hits) - len(negative_hits)   # -9
per_100 = net / len(first_tokens) * 100         # -2.96
```

The general list scores 42 of the 46 statements below zero



Low -6.51 on 2024-09-18. High 5.06 on 2026-09-16. Mean -3.88.

Keep the count beside every average

On 79 tokens a net of 4 is 5.06 per 100, and one hit moves the score by 1.27. On 304 tokens a net of -9 is -2.96, and one hit moves it by 0.33.

`=SUMPRODUCT((LEN(C2) - LEN(SUBSTITUTE(LOWER(C2), $F$2:$F$15, ""))) / LEN($F$2:$F$15))` in D2: 6 pieces of text, against 5 whole tokens

`=COUNTIF(C2:C47, "*restrictive*")` returns 4 `=(D2 - E2) / H2 * 100` net per 100 words, with the word count in H2

Where a word list goes wrong

38.7%

of all 838 hits are the one word inflation: 324 hits

74.9%

of all hits come from seven technical terms: 628 of 838

43 of 46

statements hold one sentence with inflation, a negative hit each time

Seven listed words that are technical terms here, and their hits

inflation	risks	support	pressures	unemployment	strong	restrictive
324	121	66	50	40	23	4

The same text, with and without the seven

General list	Mean	Below zero	2022-07-27
All 28 words	-3.88	42 of 46	-6.00
Without the seven	0.51	14 of 46	-0.50

Negation: not is a stopword

It is removed before any word is matched. One sentence that holds it gives one positive hit, confidence, and one negative hit, inflation: the same two hits with or without the word.

```
seven_hits = sum([positive_counts[word] + negative_counts[word]
                  for word in domain_terms])   # 628
print('not' in stop_words)                     # True
```

A lexicon score is a statement about a text and a list.

`=COUNTIF(C2:C47, "*restrictive*")` returns 4: four statements hold the word, and four sentences hold the token

`=(LEN(C2) - LEN(SUBSTITUTE(LOWER(C2), "inflation", ""))) / LEN("inflation")` times inflation appears in the statement in row 2

VADER: a published general lexicon

What VADER is

A published lexicon that ships with nltk: 7,502 entries, each with a rating, and rules for the words around them. Its authors' page says the ratings were given by human raters on a scale from -4 to 4.

The same page describes it as "a lexicon and rule-based sentiment analysis tool that is specifically attuned to sentiments expressed in social media", and says "It is fully open-sourced under the [MIT License]".

Citation, as nltk's documentation asks: Hutto, C.J. & Gilbert, E.E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Eighth International Conference on Weblogs and Social Media (ICWSM-14). Ann Arbor, MI, June 2014.

```
from nltk.sentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
analyzer.lexicon.get('inflation', 'not in the lexicon')
```

In Excel: sheet lexicon, words in A, ratings in B

`=XLOOKUP("strong", lexicon!A:A, lexicon!B:B)` returns 2.3

`=XLOOKUP("inflation", lexicon!A:A, lexicon!B:B, 0)` returns 0

What the lexicon holds, and what it does not

Everyday words	Rating	Words of this text	Rating
good	1.9	inflation	not in the lexicon
bad	-2.5	restrictive	not in the lexicon
strong	2.3	securities	1.2
weak	-1.9	treasury	0.8
support	1.7	credit	1.6
risks	-1.1	debt	-1.5
unemployment	-1.9	Entries in all	7,502

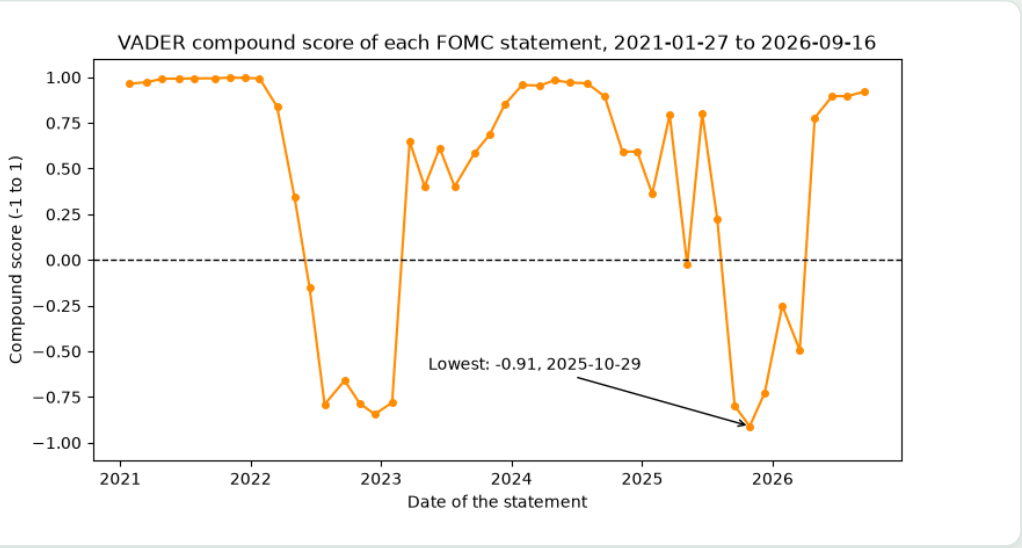
The vocabulary problem has moved

inflation, the word behind 324 hits of our own list, counts for nothing here. securities, treasury, credit, and debt are rated as everyday words. In these statements they are names of instruments.

```
analyzer.polarity_scores(sentence)   # four numbers
# neg 0.267, neu 0.428, pos 0.305, compound 0.1779
```

VADER takes the text as published

VADER compound: 15 statements above 0.9, 12 below zero



Lowest -0.91 on 2025-10-29. Highest 0.9975 on 2021-11-03.

The first nine statements are all scored between 0.96 and 1.00. A statement holds many rated words, and the normalising step presses any large sum close to 1 or -1. On a long text the compound tells little more than the sign.

General list: 42 below zero. VADER: 12. Same text, different list.

Raw text against cleaned text: the three largest gaps

Statement	As published	Cleaned
2025-06-18	0.8020	0.4404
2023-02-01	-0.7814	-0.4455
2025-09-17	-0.7992	-0.5574

41 of the 46 statement scores differ. 8 differ by more than 0.1. The largest gap is 0.36.

One sentence of the statement dated 2021-07-28, scored twice

```
as published: 0.0037  →  cleaned: 0.1531
```

The rated words are the same in both. not and but are stopwords, so the cleaning removed them, and VADER's rules for negation and for but had nothing left to work on.

```
statements['vader'] = statements['text'].apply(vader_score)
# give each tool the input it was built for: raw text
```

In Excel a lookup returns a rating. The rules that adjust it are code.

Sentence by sentence

1,198

pieces from 46 statements, cut at each full stop and space

670

pieces scored exactly 0: no rated word, or the rated words cancel

312

different pieces in the whole file

The rated words behind four sentence scores

Sentence of	Compound	Rated words, each with its rating
2024-03-20	0.8126	gained 1.6, greater 1.5, confidence 2.3
2024-03-20	-0.5106	uncertain -1.2, risks -1.1
2024-03-20	0.4019	treasury 0.8, securities 1.2, debt -1.5, securities 1.2
2021-06-16	0.891	progress 1.8, strong 2.3, support 1.7, strengthened 1.8

A rule for not, with a short reach

The sentence scored 0.8126 holds the word not. VADER has a rule for not, and it reaches only the few words that follow it. The three rated words are further along.

Names of instruments, rated

treasury, securities, and debt are names of instruments here. VADER reads mortgage-backed as one token, with no rating. A sentence scored 0.8779 has six rated words: three are treasury and securities twice.

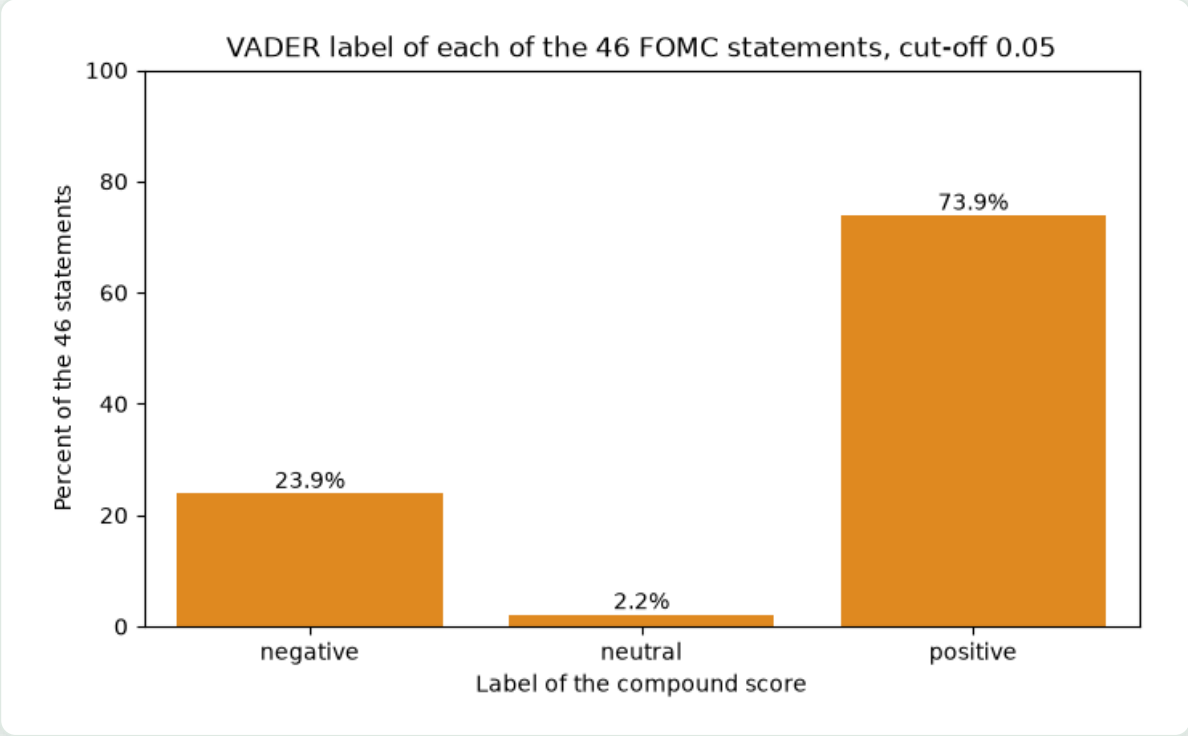
```
sentences['compound'] = sentences['sentence'].apply(vader_score)
tokens = [token.lower() for token in split_text.words_and_emoticons]
[(token, analyzer.lexicon[token]) for token in tokens if token in analyzer.lexicon]
```

`=XLOOKUP("strong", lexicon!A:A, lexicon!B:B)` returns 2.3. A lookup gives a rating. The rules that adjust the ratings are code.

A high score and a low score come from different kinds of word: not two ends of one measured thing.

A score becomes a label

VADER label of each statement, cut-off 0.05



In Excel: =IF(E2>=0.05, "positive", IF(E2<=-0.05, "negative", "neutral"))

The same scores, five cut-offs: statements by label

Cut-off	Negative	Neutral	Positive
0.05	11	1	34
0.25	10	3	33
0.5	8	9	29
0.75	6	17	23
0.9	1	30	15

```
def label_score(compound, cutoff=0.05):
    if compound >= cutoff: return 'positive'
    elif compound <= -cutoff: return 'negative'
    return 'neutral'
```

A label count depends on the cut-off as much as on the text. With no label from a person, no cut-off can be called right.

A word list built for this text

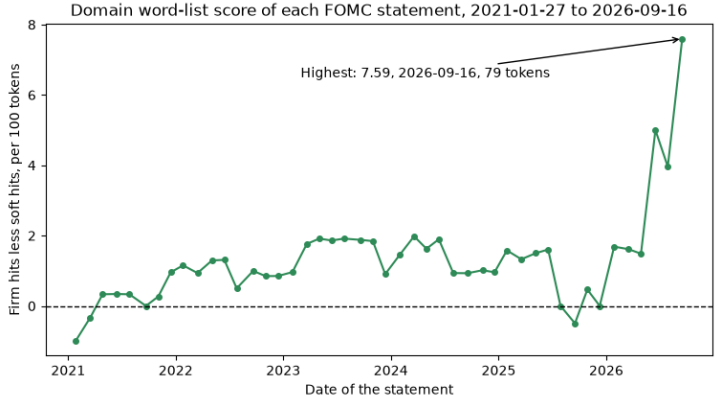
Firm: 10 words, 133 hits

Firm word	Hits	Firm word	Hits
expanding	12	robust	12
expand	11	strengthened	2
expanded	2	picked	2
solid	26	resilient	8
strong	23	gains	35

Soft: 7 words, 26 hits

Soft word	Hits	Soft word	Hits
slowed	8	weakness	1
moderated	7	weaker	1
declined	5	softened	1
weak	3	All seven	26

43 statements between -0.99 and 1.98



One teaching list among many

Which words go on it, and in which group, is a judgement. Another analyst would choose differently, and the scores would differ.

Date	Tokens	Firm	Score
2026-04-29	202	3	1.49
2026-06-17	80	4	5.0

2024-09-18 | Job gains have slowed, and the unemployment rate has moved up but remains low

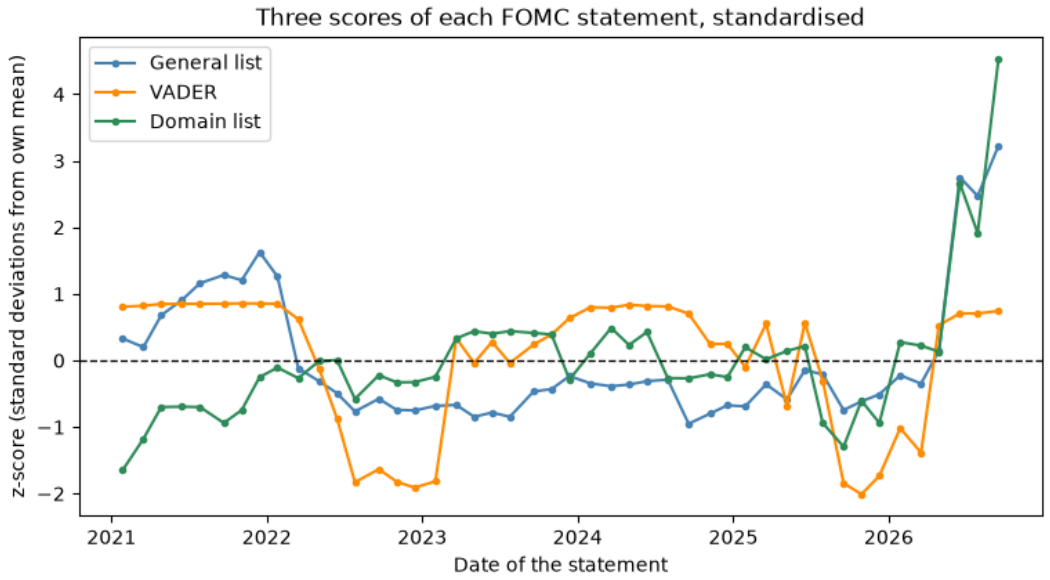
`gains: firm hit, +1` → `slowed: soft hit, -1` → `adds 0 to the net`

```
def domain_score(tokens):
    return list_score(tokens, firm_words, soft_words)
```

```
=SUMPRODUCT((LEN(C2) - LEN(SUBSTITUTE(LOWER(C2), $J$2:$J$11, ""))) / LEN($J$2:$J$11))
 - SUMPRODUCT((LEN(C2) - LEN(SUBSTITUTE(LOWER(C2), $K$2:$K$8, ""))) / LEN($K$2:$K$8))
```

Firm words in J2:J11, soft in K2:K8. Pieces of text: gains is inside against.

The three scores compared




```
z = (scores - scores.mean()) / scores.std()
print(scores.corr().round(2))
```

`=STANDARDIZE(D2, AVERAGE($D$2:$D$47), STDEV.S($D$2:$D$47))` returns 0.33

`=CORREL(D2:D47, E2:E47)` returns 0.48, with VADER in column E

Correlations of the three scores

Score	general	vader	domain
general	1.00	0.48	0.51
vader	0.48	1.00	0.21
domain	0.51	0.21	1.00

Three scales: each score over the 46 statements

Score	Mean	Lowest	Highest	Below zero
General list	-3.88	-6.51	5.06	42
VADER	0.43	-0.91	0.9975	12
Domain list	1.3	-0.99	7.59	3

z-scores: every column has mean 0 and standard deviation 1.

Mean gap, highest to lowest z-score: 1.34. On 2026-09-16: 3.79.

General and domain moved the same way in 21 of 45 changes.

Three lists, one text, three answers.

One cautious look outside the text

Score changes against same-day yield changes: all 45 points

Change in score	With 2 Yr change	With 10 Yr change
general	0.19	0.07
vader	-0.06	-0.03
domain	0.27	0.16

Without the three shortest statements: 42 points

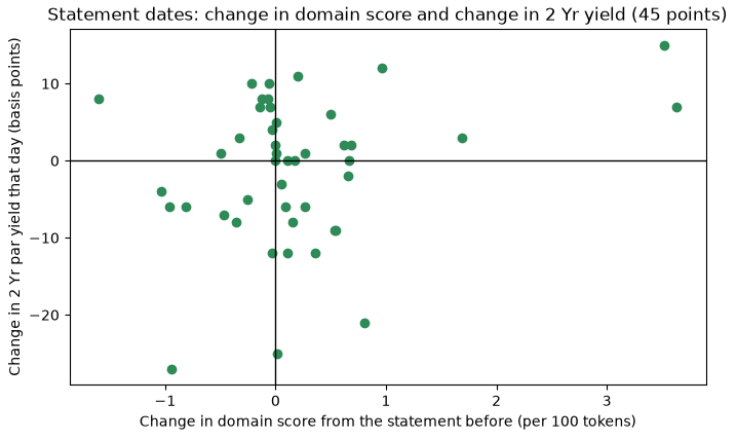
Change in score	With 2 Yr change	With 10 Yr change
general	-0.07	-0.11
vader	-0.07	-0.04
domain	0.07	0.10

The 0.27 and 0.19 rested on the three short statements.

```
joined = changes.join(yield_changes, how='left')
joined.corr()   # 46 rows, 45 of them complete
```

`=XLOOKUP(A2, yields!$A$2:$A$2941, yields!$P$2:$P$2941)` yield change

`=CORREL(F3:F47, H3:H47)` score changes in F, yield changes in H



43 of 45 points lie between -1.6 and 1.69. Two are at 3.51 and 3.63.

What these numbers are, and are not

A few dozen points. A correlation on so few moves a great deal when a handful are added or removed.

One of many things on those days. Nothing in this file separates one from another.

A par yield is a once-a-day figure. The change covers a whole day.

A correlation in this file describes this file. It is not a rule, and it says nothing about a cause.

Limits of a lexicon score

A lexicon score is a count of chosen words, divided by a length. Every part is a choice: which words, which group, what counts as a token, and what to divide by.

A lexicon score is not	What the notebook found
The meaning of a text	A word is counted the same whatever it describes: one sentence with inflation is in 43 statements
Grammar	not was removed before matching, and VADER's rule for it reaches only the few words after it
Independent of the list	The same 46 texts: 42 statements below zero on one list, 12 on a second, 3 on a third
Stable on short texts	The three shortest statements have the three highest scores on two of the three lists
A measure outside the text	Correlations near zero with same-day yield changes, on 45 points

A count of words can still be useful

It is fast

It is the same every time it is run

Every number can be traced to the words behind it

```
score = (plus - minus) / len(tokens) * 100
```

```
=(D2 - E2) / H2 * 100
```
hits in D2 and E2, word count in H2

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Score a text with a word list, and print the words that hit
- Find which words drive a score, and read the sentences behind it
- Use a published lexicon, VADER, on the raw text it was built for
- Compare scores with z-scores and a correlation
- Report a correlation with its number of points and its limits

Week 4, Day 4 From Notebook to Module. The cleaning routine becomes a file of functions. The Course Project follows on Day 5